When to use each probe

When should you use a startup probe?

Startup probes are useful for Pods that have containers that take a long time to come into service. Rather than set a long liveness interval, you can configure a separate configuration for probing the container as it starts up, allowing a time longer than the liveness interval would allow.

If your container usually starts in more than $initialDelaySeconds + failureThreshold \times periodSeconds$, you should specify a startup probe that checks the same endpoint as the liveness probe. The default for periodSeconds is 10s. You should then set its failureThreshold high enough to allow the container to start, without changing the default values of the liveness probe. This helps to protect against deadlocks.

When should you use a liveness probe?

If the process in your container is able to crash on its own whenever it encounters an issue or becomes unhealthy, you do not necessarily need a liveness probe; the kubelet will automatically perform the correct action in accordance with the Pod's restartPolicy.

If you'd like your container to be killed and restarted if a probe fails, then specify a liveness probe, and specify a restartPolicy of Always or OnFailure.

A common pattern for liveness probes is to use the same low-cost HTTP endpoint as for readiness probes, but with a higher failureThreshold. This ensures that the pod is observed as not-ready for some period of time before it is hard killed.

When should you use a readiness probe?

To start sending traffic to a Pod only when a probe succeeds, specify a readiness probe. The readiness probe might be the same as the liveness probe, but the existence of the readiness probe in the spec means that the Pod will start without receiving any traffic and only start receiving traffic after the probe starts succeeding.

You can also use a readiness probe to let a container take itself down for maintenance, by checking an endpoint specific to readiness that is different from the liveness probe.

When your app has a strict dependency on back-end services, you can implement both a liveness and a readiness probe. The liveness probe passes when the app itself is healthy, but the readiness probe additionally checks that each required back-end service is available. This helps you avoid directing traffic to Pods that can only respond with error messages.

For containers that need to work on loading large data, configuration files, or migrations during startup, consider using a startup probe. However, if you want to detect the difference between an app that has failed and an app that is still processing its startup data, you might prefer a readiness probe

When to Use Each Kubernetes Probe

Probe	Main Question	What it Controls	Typical Use
Startup Probe	"Has my application finished starting?"	Delays liveness/readiness checks until startup succeeds	Slow-starting applications
Liveness Probe	"Is my application still alive?"	Container restart	Deadlocks, hung applications
Readiness Probe	"Can my application receive traffic?"	Traffic routing	Application temporarily unable to serve requests

1. Startup Probe — Use for Slow-Starting Applications

Use a **startup probe** when the application needs significant time to become operational.

For example:

- Java/Spring Boot application takes 90 seconds to start
- Application loads a large configuration file
- Application loads a large dataset into memory
- Database migrations execute during startup
- Application performs initialization tasks

Without a startup probe, a liveness probe may start checking too early and Kubernetes could repeatedly restart the container before it finishes starting.

Key behavior:

Container starts

↓

Startup Probe

↓

Success?

/ \

No Yes

↓ ↓

Retry Startup probe stops

↓

Liveness + Readiness

A startup probe should generally allow enough time for the application to complete its **worst-case normal startup**.

For example:

```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  periodSeconds: 10
  failureThreshold: 30
```

This gives approximately:

10 × 30 = 300 seconds

of startup-probe attempts.

Important: While the startup probe has not succeeded, Kubernetes does not run the container's liveness or readiness probes.

2. Liveness Probe — Use for Hung or Deadlocked Applications

Use a **liveness probe** when an application can remain running as a process but is no longer functioning correctly.

For example:

Application process

↓

Still running

↓

BUT DEADLOCKED

↓

Liveness probe fails

↓

Kubelet restarts container

Typical situations:

- Application deadlock

- Application becomes stuck
- Internal thread failure
- Application stops responding
- Application enters an unrecoverable state

Example:

```
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
  periodSeconds: 10
  failureThreshold: 3
```

If the application fails the probe repeatedly, the kubelet restarts the container.

Important distinction

A liveness probe is **not primarily for checking whether the application can serve user traffic**.

That is the job of the readiness probe.

3. Readiness Probe — Use for Traffic Control

Use a **readiness probe** when the application is running but may not currently be capable of handling requests.

For example:

Pod Running

↓

Application starting

↓

Readiness = FAIL

↓

Service does NOT send traffic

↓

Application becomes ready

↓

Readiness = SUCCESS

↓

Service sends traffic

Typical situations:

- Application is still initializing
- Database connection is temporarily unavailable
- Required backend service is unavailable
- Application is performing maintenance
- Application temporarily wants to stop receiving traffic

Example:

```
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  periodSeconds: 5
```

If readiness fails:

The container is NOT restarted.

Instead, Kubernetes removes the Pod from the Service's ready endpoints, so traffic is not sent to it.

The Most Important Difference

Consider a Pod running a Java application.

Scenario 1 — Application is starting

Application starting

↓

Startup Probe = FAIL

↓

Keep waiting

↓

Application starts

↓

Startup Probe = SUCCESS

Use **Startup Probe**.

Scenario 2 — Application is completely stuck

Application running

↓

Application becomes deadlocked

↓

Liveness Probe = FAIL

↓

Container restarted

Use **Liveness Probe**.

Scenario 3 — Application is healthy but backend is unavailable

Application running

↓

Database unavailable

↓

Readiness Probe = FAIL

↓

Stop sending traffic

↓

Database available

↓

Readiness = SUCCESS

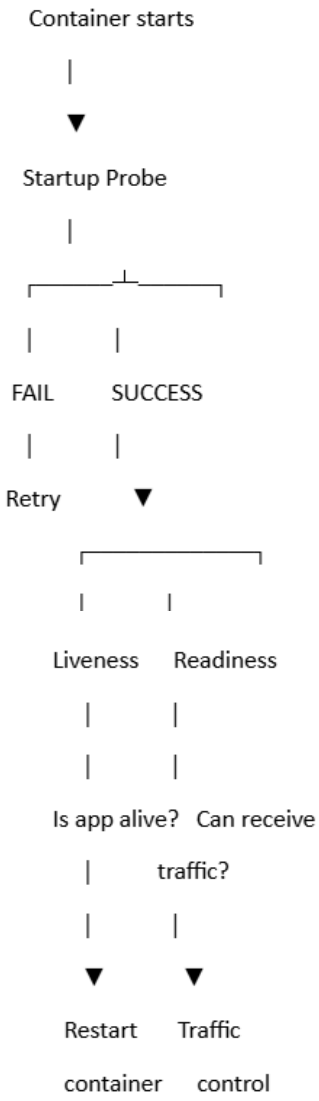
↓

Traffic resumes

Use **Readiness Probe**.

Can You Use All Three?

Yes. In production applications, it is common to use all three when their purposes are clearly separated.



Easy way for students to remember

Startup = "Can I start?"
Liveness = "Am I alive?"
Readiness = "Can I receive traffic?"

Or:

Startup → Starting?

Liveness → Alive?

Readiness → Ready?

One important production rule

Don't make the probes unnecessarily dependent on each other.

For example, if your application has:

/health/live

/health/ready

A good design is often:

Liveness:

Checks application itself

Readiness:

Checks whether application can serve traffic

+ optionally required dependencies

Startup:

Checks whether application has completed startup

This prevents a temporary database outage from unnecessarily causing Kubernetes to **restart healthy application containers**.